Week 12: Evaluation

AI Development Course

Evidence, not impressions

Tested with anthropic 1.13.0 and claude-haiku-5-5 (real evaluation runs), 13 October 2026. Jev numbers from the course's advanced/jev tutorial. A signed-in session, GitHub, and Mac and Windows laptops are being tested.

Today

- Why "it looked right" isn't evidence
- The parts of an evaluation
- Case study: Jev
- The deal screener's 20-case evaluation, run live
- Your own test set, script, misses and held-out score
- The honesty rules

"It looked right"

- You tried easy cases
- You remember the hits better than the misses
- You can't compare this version with the last one

A few good answers tell you less than they feel like.

The parts of an evaluation

1. A **test set**, including hard cases
2. **Expected answers**, written **before** any run
3. A **scoring rule**: exact, within a tolerance, the right category
4. A **score**, and the list of **misses**

Jev: small tests look perfect

- 8 easy support tickets: **8 of 8** right, all at confidence 1.00
- A refund policy, 8 cases: **7 of 8**
- The miss: $100 on day 31, "send to a manager" instead of deny, **97% confident**

Jev: 300 harder cases

How	Right
Jev alone	211 of 300 (70%)
Facts prepared first	236 of 300 (79%)
Code applies the rules, Jev reads the message (all 36 messages read right)	300 of 300
Jev alone, within 3 days of a window's edge	63 of 129 (49%)

300 generated cases from 36 messages, one run, written by the author: not a general accuracy.

Three lessons from Jev

- **Errors cluster where the rules bite**
- **Confidence isn't correctness**: kept only at 90% or more confident, 174 of 179 right, 5 confident misses left
- **Small test sets hide things**

How many cases?

- Anthropic's guide gives no minimum; its examples use 50 to 10,000
- "Prioritize volume over quality"
- This week: **20**, each answer written and checked by you
- 20 cases show **big failures and patterns**, not a precise rate
- One case = 5% of the score: 17 vs 18 out of 20 could be luck

The deal screener's test set

20 made-up listings:

- clean, messy, numbers missing
- traps: a yearly rent, a price range, two prices
- extra numbers that aren't inputs, "no HOA" in words

Right answers committed **before** the first run. Rule: within $1; `null` only if not stated. 5 held out.

Run it

```
python3 -m dealscreener.evaluate              # 15 dev cases
python3 -m dealscreener.evaluate --held-out   # 5 held-out, once
python3 -m dealscreener.evaluate --all        # all 20
```

One line per case, then the score, the misses and the cost. Saved in `evals/results/`.

The real results

Run	Score	Cost
Dev, run 1	75 of 75 (100%)	$0.0017
Dev, run 2	75 of 75 (100%)	$0.0016
Held out, once	24 of 25 (96%)	$0.0005
All 20 (held-out cases again)	100 of 100 (100%)	$0.0023

The one miss

> "The seller will consider offers between $180,000 and $195,000."

- Expected: `null` (no single price)
- Held-out run: **187500**, the middle: a guess
- All-20 run, same listing: `null`

Same input, two answers. Run it more than once.

What 15 perfect dev cases mean

- No big failure: monthly tax, yearly rent, quarterly HOA, missing numbers
- **Not** "100% accurate"
- Maybe the set is **too easy**: add harder cases (two rents, a weekly figure, a typo)

The honesty rules

1. Write the right answers **first**, and commit them
2. **Never** change the test set to raise the score
3. Fix a wrong answer only with a **written reason**
4. Run the **held-out** 5 once, after your one change
5. Report the **misses**, and what the score can't show

Your test set

Kind	How many
Clean, easy	4 or 5
Messy	4 or 5
Missing or impossible ("not stated", "ask")	3 or 4
Traps	3 or 4
Edge cases	2 or 3

Course-safe data. Scoring rule written first.

Then

1. Branch `eval`; commit the test set **alone**
2. `/superpowers:writing-plans` for the script, with the guardrails
3. Dev cases twice, in a second window (never the Claude Code one); read every miss
4. One change; dev again
5. Held-out, once
6. Review, pull request (no `.env`, no key in any file), **Create a merge commit**, pull

The assignment

Criterion	Points
Test set: 20 cases, reasons, 5 held out, scoring rule, committed first	3
Merged script: offline tests, score, misses, cost, results saved, no key	3
Report: two dev scores, three misses and corrections, one change and held-out score, what it shows	4

Next week: security and responsible use.